Pensar en Componentes

Deja de construir páginas y empieza a construir piezas. Crea tu proyecto con Vite, escribe JSX, arma componentes y pásales datos con props. Todo el código se ejecutó con React 19.3 y Vite 8.3: lo que ves en estas hojas es lo que verás en tu pantalla.



4
subniveles, de Vite a las props

52
estaciones con código real, casos, misiones y retos

800
puntos de experiencia (XP) en juego para tu equipo

5
rangos, de aprendiz de piezas a pensador/a en componentes

Equipo _____ Curso _____

Antes de empezar: tu ruta de constructor/a

Bienvenida

Este cuaderno es el nivel 1 de tu ruta de React: pensar en componentes. Tu equipo construye en React la pantalla de **Arena U**, la app del Club Pixel para la Copa Pixel, el torneo de videojuegos del campus. En cada subnivel lees un ejemplo resuelto y probado, lo desarmas pieza por pieza, preguntas a la IA sin que te dé la solución y resuelves un problema real paso a paso. **No hay respuestas al final**: las construye tu equipo.

LA RUTA COMPLETA

Nivel	Tema	La pregunta que vas a responder	Tiempo	Botín
1.1	**React y Vite: tu primer proyecto**	¿Cómo se crea hoy un proyecto React?	60 min	200 XP
1.2	**JSX: HTML dentro de JavaScript**	¿Qué reglas sigue el marcado en React?	60 min	200 XP
1.3	**Componentes: funciones que devuelven interfaz**	¿Cómo se parte una pantalla en piezas?	60 min	200 XP
1.4	**Props: datos de padre a hijo**	¿Cómo recibe datos una pieza?	60 min	200 XP
	Ruta completa	**4 preguntas, una historia: Arena U**	**240 min**	**800 XP**

ROLES DEL EQUIPO · ROTAN EN CADA NIVEL

Piloto
Escribe el código y lo ejecuta. Piensa en voz alta mientras teclea.

Copiloto
Lee cada línea, detecta errores y pregunta a la IA por partes del código.

Diseñador/a
Cuida cómo se ve y se usa la pantalla: textos, contraste y piezas que se repiten.

Guardián/a
Cuida el tiempo, el marcador de XP y que todos participen.

RANGOS · SUMA TUS XP DE TODOS LOS NIVELES

RANGO 1
Aprendiz de piezas
0–159 XP

RANGO 2
Armador/a de JSX
160–319 XP

RANGO 3
Constructor/a de componentes
320–479 XP

RANGO 4
Mensajero/a de props
480–639 XP

RANGO 5
Pensador/a en componentes
640–800 XP

LEYENDA DE ESTACIONES · QUÉ HACES EN CADA UNA

- **E01 · ¿Sabías que?** Datos reales que sorprenden.
- **E02 · Explicación** La idea en bloques cortos.
- **E03 · Código ejemplo** Código real de React 19.3 y Vite 8.
- **E04 · Anatomía** Cada parte del código, explicada.
- **E05 · Pregunta a la IA** Por líneas, no por soluciones.
- **E06 · Buenas prácticas** Evita esto, mejor así.
- **E07 · El caso** Un episodio de Arena U.
- **E08 · Tips de diseño** UX y UI sobre un boceto.
- **E09 · Misión** Problema real, paso a paso.
- **E10 · Comodines** Quita opciones o usa pista.
- **E11 · Evaluación** Seis preguntas sin mirar.
- **E12 · Tendencias 2026** Industria, datos e IA.
- **E13 · Voz del diseñador** Una frase y la tuya.
- **Cierre · Cierre** Semáforo, marcador y 60 s.
- **XP · Botín** 200 XP por nivel.

RELOJ SUGERIDO PARA UN NIVEL DE 60 MINUTOS

E01	E02	E03	E04	E05	E06	E07	E08	E09	E10	E11	E12	E13	Cierre
3 min	6 min	5 min	4 min	4 min	3 min	3 min	3 min	14 min	1 min	6 min	3 min	2 min	3 min

La misión (E09) es la estación más larga: es donde más se aprende. Si una estación se alarga, recorten en la siguiente, nunca en la misión.

¿QUÉ ESPERA NUESTRO EQUIPO DE ESTE TALLER?

Cómo está diseñado cada nivel

Guía

Las estaciones no están en cualquier orden. Siguen el camino que recorre tu cerebro cuando aprende a programar: primero se engancha, luego comprende con un ejemplo resuelto, aplica en un caso real, comprueba sin mirar, conecta con el frontend de hoy y, al final, reflexiona.

EL ARCO DE APRENDIZAJE DE CADA NIVEL

Engancha

E01
Un dato sorprendente abre la curiosidad: el cerebro presta atención cuando nota que le falta una pieza.

Comprende

E02 · E03 · E04 · E05
Primero la idea; luego un ejemplo resuelto que lees pieza por pieza y aclaras con la IA.

Aplica

E06 · E07 · E08 · E09
Con buenas prácticas y tips de diseño resuelves un problema real de Arena U, paso a paso.

Comprueba

E10 · E11
Recordar sin mirar fija lo aprendido. Los comodines te dejan pedir ayuda sin miedo.

Conecta

E12
Llevas lo aprendido al frontend real de hoy: datos, empresas e impacto de la IA.

Reflexiona

E13 · Cierre
Miras tu trabajo con ojos de diseñador, evalúas tu seguridad y lo explicas a otro equipo.

ANATOMÍA DE UN NIVEL · 8 HOJAS, 13 ESTACIONES

Apertura Misión, recuerdo, metas y botín	**Hoja 1** E01 ¿Sabías que? · E02 Explicación	**Hoja 2** E03 Código ejemplo · E04 Anatomía	**Hoja 3** E05 Pregunta a la IA · E06 Buenas prácticas
Hoja 4 E07 El caso · E08 Tips de diseño	**Hoja 5** E09 Misión paso a paso	**Hoja 6** E10 Comodines · E11 Evaluación	**Hoja 7** E12 Tendencias 2026 · E13 Diseñador · Cierre

¿POR QUÉ ESTE CUADERNO FUNCIONA ASÍ?

Recuerda sin mirar
Cada apertura te pide recordar el nivel anterior. Recuperar la información la fija mejor que releerla.

Curiosidad primero
Empiezas con datos que sorprenden. Una pregunta abierta motiva más que una respuesta dada.

Ejemplo resuelto
Ves código completo y explicado antes de escribir el tuyo: así la memoria no se satura.

Predice y compara
Antes de ejecutar, adivinas qué pasará. Equivocarte y comparar también enseña.

Ayuda que se retira
En la misión el código llega con huecos: tú completas. Cada nivel, menos andamio.

Palabra, código e imagen
Pantallas dibujadas, íconos y colores acompañan el texto: el cerebro guarda mejor lo que ve.

Retroalimentación rápida
XP, semáforo y marcador te dicen al instante cómo vas y qué repasar.

Programar en pareja
Piloto y copiloto se turnan: uno escribe, el otro lee, pregunta y revisa.

Enseñar para aprender
Explicar en 60 segundos obliga a ordenar lo que sabes con tus palabras.

CÓDIGO DE COLOR · CADA COLOR TIENE UN SIGNIFICADO

- **1.1 Celeste** — Inicio y claridad
- **1.2 Violeta** — Lenguaje e imaginación
- **1.3 Esmeralda** — Construcción y crecimiento
- **1.4 Naranja** — Comunicación y energía
- **XP Oro** — Recompensa y logro

REGLAS DE ORO DEL EQUIPO

- [ ] Las respuestas se construyen en equipo: este cuaderno no trae soluciones.
- [ ] Todo código se ejecuta y se mira en pantalla antes de darlo por bueno.
- [ ] Piloto y copiloto cambian de puesto en cada paso de la misión.
- [ ] A la IA se le pregunta por una línea o un concepto, nunca por la misión resuelta.
- [ ] Cada comodín se anota en el marcador, con su costo o su bono.
- [ ] Nunca compartimos datos personales con la IA ni con otros equipos.

NOMBRE Y LEMA DE NUESTRO EQUIPO

Integrante 1 | Integrante 2 | Integrante 3 | Integrante 4

NIVEL 1.1 · DEL COMANDO A LA PANTALLA

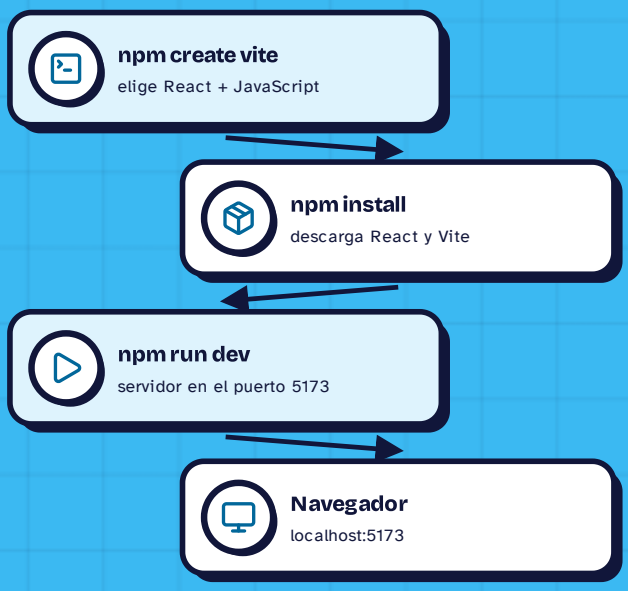
React y Vite: tu primer proyecto en 2026

IDEA CLAVE React = biblioteca para construir interfaces con componentes · Vite = herramienta que crea, sirve y empaqueta el proyecto

TU MISIÓN

El Club Pixel quiere rehacer la app de Arena U con React, pero nadie sabe por dónde empezar: unos siguen tutoriales viejos y otros copian carpetas ajenas. Tu equipo debe crear el proyecto como se hace en 2026, con **Vite**, y ver su primera pantalla en el navegador.

DEL COMANDO A LA PANTALLA



RECUERDA SIN MIRAR

Sin buscar: ¿qué tres lenguajes usa una página web y qué hace cada uno?

E01 ¿Sabías que? · E02 Explicación · E03 Código ejemplo · E04 Anatomía · E05 Pregunta a la IA · E06 Buenas prácticas · E07 El caso · E08 Tips de diseño · E09 Misión · E10 Comodines · E11 Evaluación · E12 Tendencias · E13 Voz del diseñador

ENGANCHA · COMPRENDE · APLICA · COMPRUEBA · CONECTA · REFLEXIONA

AL TERMINAR ESTE NIVEL PODRÁS

- [ ] Explicar qué es React y qué papel cumple Vite.
- [ ] Crear un proyecto React con create-vite, paso a paso.
- [ ] Reconocer los archivos clave: index.html, main.jsx y App.jsx.
- [ ] Ejecutar el servidor de desarrollo y ver los cambios al guardar.

MI META PERSONAL PARA ESTE NIVEL

BOTÍN DEL NIVEL

Reto flash de ¿Sabías que?	+10
Predice el código	+15
Pregunta a la IA verificada	+20
Reto de diseño	+10
Misión paso a paso	+60
Evaluación	+40
Tendencias 2026	+20
Voz del diseñador	+10
Explícalo en 60 segundos	+15
Total del nivel	**200 XP**

E01 ¿Sabías que?

ENGANCHA +10 XP

NACIÓ EN FACEBOOK

2013

React se presentó como código abierto en la JSConf US de mayo de 2013. Lo creó Jordan Walke, ingeniero de Facebook.

Fuente: Wikipedia, «React (software)», 2026.

ADIÓS, CRA

2025

el 14 de febrero, React dejó de recomendar Create React App para apps nuevas: propone frameworks o herramientas como Vite.

Fuente: React Blog, «Sunsetting Create React App», 2025.

RÁPIDO EN FRANCÉS

65 M

de descargas por semana tiene Vite, cuyo nombre significa «rápido» en francés.

Fuente: Vite Blog, «Announcing Vite 8», 2026; vite.dev.

RETO FLASH · 60 S — En 60 segundos, escriban las partes que se repiten en la pantalla de una app que usen (botones, tarjetas, menús…). Gana el equipo que encuentre más piezas reutilizables. +10 XP

E02 Explicación breve: React pone las piezas, Vite el taller

COMPRENDE

DEFINICIÓN DE TRABAJO

React es una biblioteca de JavaScript para construir interfaces con componentes: piezas reutilizables que unen marcado, estilo y lógica. Vite es la herramienta que crea el proyecto, lo sirve mientras programas y lo empaqueta para publicarlo.

En lugar de construir «páginas» completas, en React armas la interfaz con **componentes**, como piezas que encajan: un encabezado, una tarjeta de equipo, un marcador. Cada pieza se escribe una vez y se usa muchas veces.

React no trae servidor ni empaquetador. Para apps completas, react.dev recomienda un framework (Next.js, React Router o Expo); para aprender las bases, una herramienta como **Vite**, Parcel o Rsbuild. Vite crea el proyecto con `npm create vite@latest`.

Mientras programas, **npm run dev** levanta un servidor en **localhost:5173** que actualiza la página al guardar. Al terminar, **npm run build** genera la carpeta dist lista para publicar. Vite 8 pide Node.js 20.19+ o 22.12+.



FIG 1.1 Tres comandos llevan del proyecto vacío a la primera pantalla. El cuarto paso, mirar el navegador, nunca se salta.

Pieza	¿Qué es?	Versión (oct. 2026)	En el proyecto
Node.js	Ejecuta JavaScript fuera del navegador.	20.19+ o 22.12+	Lo pide Vite 8.
npm	Instala paquetes y corre scripts.	Viene con Node.js	`npm run dev`
Vite	Crea, sirve y empaqueta el proyecto.	8.3.3	`vite.config.js`
React	Biblioteca de componentes.	19.3.0	`App.jsx`
create-vite	Asistente que genera la plantilla.	9.2.1	`npm create vite@latest`

3 IDEAS PARA LLEVAR

1. **Piezas, no páginas** — La interfaz se arma con componentes.
2. **Vite es el taller** — Crea, sirve y empaqueta tu proyecto.
3. **Tres comandos** — create, install y dev; luego, mira el navegador.

RESÚMELO EN UNA FRASE CON TUS PALABRAS

UNA DUDA QUE ME QUEDÓ

PALABRAS CLAVE: React · Vite · Node.js · npm · create-vite · plantilla · servidor de desarrollo · localhost

E03 Código ejemplo: el proyecto Arena U, creado con Vite

COMPRENDE +15 XP

Lucía creó el proyecto el 7 oct. 2026 con create-vite 9.2.1. La terminal y los archivos son reales, copiados de la ejecución.

terminal — TERMINAL

```
$ npm create vite@latest arena-ui -- \ 1
    --template react --no-interactive 2
o  Scaffolding project in …/arena-ui...
—  Done. Now run:
  cd arena-ui
  npm install
  npm run dev
$ cd arena-ui
$ npm install 3
added 24 packages, and audited 25 packages in 14s
$ npm run dev 4
  VITE v8.3.3  ready in 294 ms
  ➜  Local:   http://localhost:5173/
```

arena-ui — CARPETAS

```
arena-ui/
├── index.html 5
├── package.json
├── vite.config.js
├── .oxlintrc.json
├── public/
│   ├── favicon.svg
│   └── icons.svg
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── App.css
    ├── index.css
    └── assets/
```

src/main.jsx — JSX

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5
6  createRoot(document.getElementById('root')).render( 6
7    <StrictMode>
8      <App /> 7
9    </StrictMode>,
10 )
```



ASÍ SE VE Así se ve en localhost:5173 tras guardar App.jsx.

src/App.jsx — JSX

```
1  export default function App() {  8
2    return (
3      <main>
4        <h1>¡Hola, Arena U!</h1>
5        <p>Copa Pixel 2026 · 16 cupos</p>
6      </main>
7    )
8  }
```

Predice antes de ejecutar +15 XP

1. Si cambias el texto del h1 y guardas, ¿tendrás que volver a ejecutar npm run dev? ¿Por qué?

2. Si borras la carpeta node_modules, ¿qué comando la recupera?

☐ ACERTAMOS ☐ CASI ☐ NOS SORPRENDIÓ

E04 Anatomía del código: pieza por pieza

COMPRENDE

#	Fragmento	Qué es	Qué hace y por qué importa
1	`npm create vite@latest`	**Crear**	Descarga create-vite y genera la carpeta arena-ui con la plantilla.
2	`--template react`	**Plantilla**	Elige React con JavaScript; sin esta opción, el asistente pregunta.
3	`npm install`	**Dependencias**	Descarga lo que pide package.json: React, Vite y más (24 paquetes).
4	`npm run dev`	**Servidor**	Ejecuta el script dev: Vite sirve la app en localhost:5173.
5	`index.html`	**Punto de entrada**	La página real: trae un div con id root y carga src/main.jsx.
6	`getElementById('root')`	**Raíz de React**	Busca ese div en index.html y dibuja dentro todo lo que haga React.
7	`<App />`	**Componente raíz**	Se dibuja dentro de StrictMode, que ayuda a encontrar errores en desarrollo.
8	`function App()`	**Tu componente**	Una función que devuelve la interfaz: aquí empiezas a programar.

E05 Pregunta a la IA por partes del código

COMPRENDE +20 XP

Pregunta por una línea, no por la solución

+20 XP SI VERIFICAN

P1 ¿Qué diferencia hay entre **React** y **Vite**? Explícalo con la analogía de las piezas y el taller.

P2 ¿Qué hace **createRoot(document.getElementById('root'))** en main.jsx? ¿Qué pasa si el id no coincide con el de index.html?

P3 ¿Para qué sirve **npm install** y por qué la carpeta node_modules no se sube a Git?

Regla de oro: pregunta por una línea o un concepto, nunca por la misión resuelta. Si la IA te propone **create-react-app**, recuerda que React dejó de recomendarlo en febrero de 2025: contrasta con react.dev y vite.dev.

FÓRMULA PARA PREGUNTAR POR CÓDIGO · ÚSALA EN P1, P2 Y P3

ROL	FRAGMENTO	PREGUNTA	NIVEL	LÍMITE
Actúa como mentor/a de React 19.	Pega solo la línea que no entiendes.	¿Qué hace y qué pasa si la quito?	Explícalo para alguien que empieza.	No resuelvas mi misión.

Parte del código	Qué dijo la IA (en tus palabras)	¿Coincide con la anatomía?	¿Cómo lo comprobaste?
React vs. Vite		◯ Sí ◯ En parte ◯ No	
createRoot y #root		◯ Sí ◯ En parte ◯ No	
node_modules		◯ Sí ◯ En parte ◯ No	

E06 Buenas prácticas: así escriben los pros

APLICA

Nada de create-react-app

Usa una herramienta vigente.

EVITA

```
$ npx create-react-app arena-ui
```

React dejó de recomendarlo para apps nuevas en febrero de 2025.

MEJOR ASÍ

```
$ npm create vite@latest arena-ui
```

Vite es una de las herramientas que propone react.dev para aprender las bases.

node_modules fuera de Git

Se reinstala, no se guarda.

EVITA

```
$ git add node_modules
```

Subes miles de archivos que cualquiera recrea con npm install.

MEJOR ASÍ

```
# .gitignore de la plantilla:
node_modules
dist
```

La plantilla de Vite ya trae un .gitignore que los excluye.

Guarda y mira

El servidor actualiza solo.

EVITA

```
# detener y volver a ejecutar
# npm run dev en cada cambio
```

Pierdes tiempo: mientras corre, Vite ya actualiza la página.

MEJOR ASÍ

```
# deja corriendo npm run dev,
# guarda y mira localhost:5173
```

Vite reemplaza en caliente el módulo que cambiaste.

¿QUÉ COMANDO USARÍAN PARA CREAR EL PROYECTO DE LA LIGA Y QUÉ PLANTILLA ELEGIRÍAN?

¿QUÉ ARCHIVO ABREN PRIMERO PARA CAMBIAR EL TEXTO DE LA PANTALLA? ¿POR QUÉ ESE?

E07 El caso: tres arranques, ninguno igual

APLICA · Lectura

Club Pixel · Arena U en React

CASO · ARENA U

Lucía siguió un tutorial antiguo que usaba create-react-app y pasó la tarde peleando con advertencias de instalación. Samuel copió la carpeta del proyecto de otro club, sin node_modules, y no sabía qué instalar ni en qué orden. Tomás, el diseñador, quería empezar por los colores antes de que existiera una sola pantalla.

Valeria, que ya tenía experiencia con proyectos web, propuso un acuerdo: un solo comando, la plantilla oficial de Vite y una primera pantalla que diga «¡Hola, Arena U!» antes de diseñar nada. En diez minutos, los tres veían lo mismo en localhost:5173.

> **«Primero que arranque y se vea; después lo hacemos bonito.»** — Valeria, Club Pixel

ARRANQUES	COMANDOS	PUERTO	MINUTOS
3	3	5173	10
formas distintas de empezar	create, install y dev	donde Vite sirve la app	para ver la primera pantalla

E08 Tips de diseño: una primera pantalla clara

APLICA · +10 XP



BOCETO DE TOMÁS La primera pantalla de Arena U.

1. **Un título que orienta**
   La primera pantalla dice dónde estás y para qué sirve, sin que tengas que pensar.
   Krug, *Don't Make Me Think*, 2000.

2. **Texto que se escanea**
   El 79 % de las personas de un estudio escaneó las páginas; solo el 16 % leyó palabra por palabra.
   Nielsen, «How Users Read on the Web», 1997.

3. **Primero el celular**
   Diseñar primero para la pantalla pequeña obliga a quedarse con lo esencial.
   Wroblewski, *Mobile First*, 2011.

4. **Contraste suficiente**
   El texto normal necesita un contraste mínimo de 4,5:1 con su fondo.
   W3C, WCAG 2.2, criterio 1.4.3.

RETO DE DISEÑO · DIBUJEN LA PRIMERA PANTALLA DE SU ARENA U Y MARQUEN EL TÍTULO, EL DATO CLAVE Y LA ACCIÓN PRINCIPAL

E09 Misión: el proyecto de la liga, en vivo

APLICA +60 XP

Creen su propio proyecto con el asistente interactivo de create-vite y cambien la primera pantalla. Turnen piloto y copiloto en cada paso.

Liga Pixel · primer proyecto React

PASO A PASO

1 **Responde al asistente.** Ejecuten npm create vite@latest y anoten qué eligieron en cada pregunta (estas son las preguntas reales de create-vite 9.2.1). +15 XP

Pregunta del asistente	Su respuesta	¿Por qué?
`Project name:`		
`Select a framework:`		
`Select a variant:`		
`Which linter to use?`		
`Install with npm and start now?`		

2 **Arranca.** Completen los comandos si respondieron «No» a la última pregunta. +10 XP

terminal — TERMINAL

```
$ cd _____________
$ npm _____________
$ npm run ________
```

3 **Cambia la pantalla.** Reemplacen el contenido de App.jsx por la bienvenida de la liga. +15 XP

src/App.jsx — JSX

```jsx
export default function _________() {
  return (
    <main>
      <h1>_________________________</h1>
      <p>_________________________</p>
    </main>
  )
}
```

4 **Verifica.** Anoten lo que esperaban y lo que vieron. +20 XP

Comprobación	Lo que esperaban	Lo que vieron
Puerto del servidor		
Título en el navegador		
Al guardar, ¿se actualizó sola?		

DEFINICIÓN DE TERMINADO · MARQUEN SOLO LO QUE COMPROBARON EN PANTALLA

- [ ] El proyecto se creó con create-vite, no con create-react-app.
- [ ] npm run dev muestra la app en localhost.
- [ ] El título nuevo aparece al guardar, sin reiniciar el servidor.
- [ ] Explicamos qué hacen index.html, main.jsx y App.jsx.

Firma del copiloto: revisó y probó

E10 Comodines: quitar o usar ayuda

COMPRUEBA -5 / +10 XP

Quitar opciones -5 XP

Para la **P2**: descarta los puertos de otras herramientas; el número aparece en la salida de npm run dev.

```
Tachamos: ______ y ______
```

☐ ☐ USADO

Usar pista -5 XP

Para la **P5**: relee la anatomía (E04), filas 5 y 6.

```
Pista usada en: P___
```

☐ ☐ USADO

Otro equipo +10 AL QUE AYUDA

Expliquen su razonamiento, no la respuesta. Si el otro equipo aprende, ustedes ganan.

```
¿Cómo pensaron ______ y por qué?
```

☐ USADO

E11 Preguntas de evaluación

COMPRUEBA +40 XP

Respondan sin mirar las hojas anteriores. Después, revisen juntos y sumen 5 XP por cada respuesta bien justificada (máximo 40).

P1 Según react.dev, para aprender las bases de React desde cero, ¿qué conviene usar? OPCIÓN MÚLTIPLE

- ◯ Create React App.
- ◯ Una herramienta de construcción como Vite, Parcel o Rsbuild.
- ◯ Solo un archivo HTML sin herramientas.
- ◯ Un servidor de base de datos.

P2 ¿En qué puerto abre Vite el servidor de desarrollo por defecto? OPCIÓN MÚLTIPLE

- ◯ 3000
- ◯ 8080
- ◯ 5173
- ◯ 4200

P3 React trae su propio servidor de desarrollo; Vite solo sirve para publicar la app. V F V/F Y POR QUÉ

P4 Ordena los pasos para ver tu primera pantalla. ORDENA

☐ npm run dev ☐ Abrir localhost:5173 ☐ npm create vite@latest ☐ npm install ☐ cd arena-ui

P5 ¿Qué elemento de index.html busca esta línea? Escribe la etiqueta completa.

```
createRoot(
  document.getElementById('root'))
```

RESPUESTA CORTA

P6 Explica con tus palabras por qué el tutorial que siguió Lucía ya no era buena idea en 2026. RESPUESTA CORTA

Reto bonus · Caza el error

+5 XP DE BONO

src/main.jsx — JSX

```jsx
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.jsx'
5
6  createRoot(document.getElementById('app')).render(
7    <StrictMode>
8      <App />
9    </StrictMode>,
10 )
```

index.html no cambió: sigue teniendo `<div id="root">`. La pantalla queda en blanco y la consola del navegador dice: `Target container is not a DOM element.`

¿QUÉ LÍNEA CAUSA EL ERROR?

¿POR QUÉ REACT NO ENCUENTRA DÓNDE DIBUJAR?

ESCRIBAN LA LÍNEA CORREGIDA

E12 Tendencias 2026: React y Vite cambian de motor

CONECTA +20 XP

VITE	ROLLDOWN	LINEAR	REACT
8.3.3	**10–30×**	**46 → 6 s**	**19.3**
versión publicada el 6 oct. 2026	builds más rápidos que con Rollup	su build de producción con Vite 8	versión publicada el 9 sep. 2026

Fuentes: registro de npm (oct. 2026); Vite Blog, «Announcing Vite 8», 12 mar. 2026; React Blog, 9 sep. 2026.

Así lo hace la industria — PRÁCTICA

- **React Compiler 1.0.** Salió en octubre de 2025 y create-vite ya ofrece plantillas con el compilador.
- **Oxlint.** La plantilla React de create-vite 9 lo trae como revisor de código por defecto.
- **React 19.3.** Hace estable <ViewTransition> para animar elementos que entran, salen o cambian.
- **Frameworks.** Para apps completas, react.dev recomienda Next.js, React Router o Expo.

Impacto de la IA — 2026

- OPORTUNIDAD La IA explica un error de la terminal si le pegas la línea exacta.
- OPORTUNIDAD Pídele una lista de comprobación para tu primer proyecto y revísala.
- RIESGO Puede proponer create-react-app, que React dejó de recomendar en 2025.
- RIESGO El 66 % se frustra con soluciones de IA «casi correctas» (Stack Overflow, 2025).

TU POSTURA ¿Qué app que usas a diario crees que está hecha con componentes? ¿Qué piezas ves repetidas?

E13 ¿Qué diría un diseñador front?

REFLEXIONA +10 XP

> **¡No me hagas pensar!**
>
> SK **Steve Krug**
> Primera ley de usabilidad, *Don't Make Me Think*, 2000

TU TURNO · +10 XP

Si la primera pantalla de Arena U no dice qué es ni qué hacer, ¿qué diría un diseñador front? Escribe su frase.

Lupa del diseñador · criterio	¿Su pantalla lo cumple?	Evidencia o cambio que harían
El título dice qué es la app.	◯ Sí ◯ En parte ◯ No	
El dato clave se ve sin leer todo.	◯ Sí ◯ En parte ◯ No	
Hay una sola acción principal.	◯ Sí ◯ En parte ◯ No	

Semáforo: ¿qué tan seguro estás? ROJO · AMARILLO · VERDE

Explico qué hacen React y Vite.	🔴 🟡 🟢
Creo un proyecto con create-vite y lo ejecuto.	🔴 🟡 🟢
Reconozco index.html, main.jsx y App.jsx.	🔴 🟡 🟢

Explícalo en 60 segundos. Expliquen a otro equipo cómo pasaron de un comando a una pantalla, en tres pasos. Si el otro equipo crea su proyecto siguiendo solo su explicación, ambos ganan el bono. +15 XP

Insignia: Primer arranque. Coloréala en el Salón de insignias si tu equipo logra 140 XP o más en este nivel.

MARCADOR DEL NIVEL 1.1

XP ganados	+ ___
Comodines usados	− ___
Bono por ayudar	+ ___

TOTAL _____ /200

{} NIVEL 1.2 / 1.4

60 MIN | 200 XP

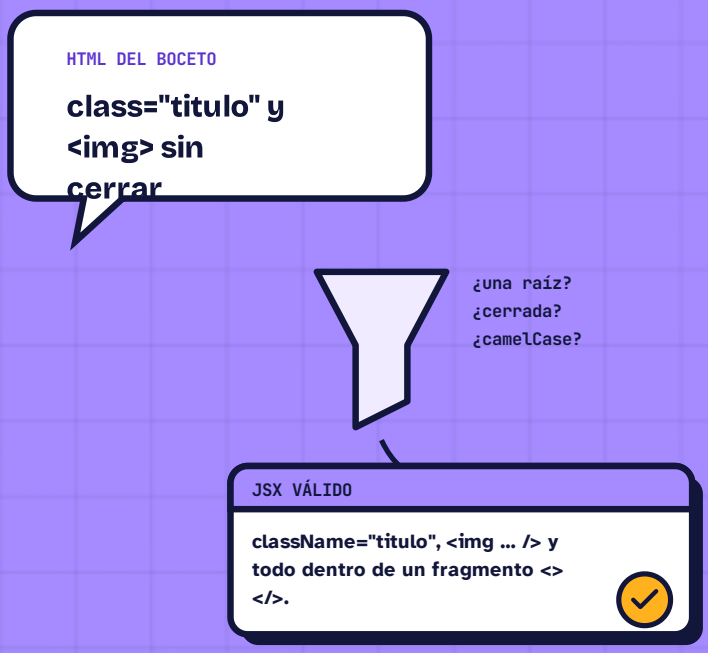
NIVEL 1.2 · MARCADO CON REGLAS PROPIAS

JSX: HTML dentro de JavaScript

IDEA CLAVE JSX = marcado parecido a HTML dentro de JavaScript: una sola raíz, etiquetas cerradas, className y {expresiones}

TU MISIÓN

Tomás pegó el HTML de su boceto directamente en un componente y Vite dejó de compilar: dos raíces, etiquetas sin cerrar y class por todas partes. Tu equipo debe traducir ese marcado a **JSX** siguiendo sus reglas.



RECUERDA SIN MIRAR

Sin mirar el nivel 1.1: ¿qué hace npm run dev y en qué dirección se abre la app?

E01 ¿Sabías que? | E02 Explicación | E03 Código ejemplo | E04 Anatomía | E05 Pregunta a la IA | E06 Buenas prácticas | E07 El caso | E08 Tips de diseño | E09 Misión | E10 Comodines | E11 Evaluación | E12 Tendencias | E13 Voz del diseñador

ENGANCHA · COMPRENDE · APLICA · COMPRUEBA · CONECTA · REFLEXIONA

AL TERMINAR ESTE NIVEL PODRÁS

- [ ] Explicar qué es JSX y en qué se convierte.
- [ ] Aplicar sus reglas: una raíz, etiquetas cerradas y camelCase.
- [ ] Insertar datos de JavaScript con llaves { }.
- [ ] Leer los errores de Vite y corregirlos.

MI META PERSONAL PARA ESTE NIVEL

BOTÍN DEL NIVEL

Reto flash de ¿Sabías que?	+10
Predice el código	+15
Pregunta a la IA verificada	+20
Reto de diseño	+10
Misión paso a paso	+60
Evaluación	+40
Tendencias 2026	+20
Voz del diseñador	+10
Explícalo en 60 segundos	+15
Total del nivel	**200 XP**

E01 ¿Sabías que?

ENGANCHA +10 XP

SIN IMPORTAR REACT

2020

React 17 trajo una nueva transformación de JSX: desde entonces se puede usar JSX sin importar React en el archivo.

Fuente: React Blog, 22 sep. 2020.

PALABRA RESERVADA

class

es palabra reservada de JavaScript. Por eso en JSX se escribe className, como la propiedad del DOM.

Fuente: react.dev, «Writing Markup with JSX», 2026.

NUEVO DUEÑO

2026

React, React Native y proyectos como JSX pasaron a la React Foundation, alojada por la Linux Foundation.

Fuente: React Blog, 24 feb. 2026.

RETO FLASH · 60 S — En 60 segundos, escriban atributos de HTML que conozcan (class, for, onclick…) y adivinen cómo se escriben en JSX. Gana el equipo con más aciertos. +10 XP

E02 Explicación breve: las reglas del JSX

COMPRENDE

DEFINICIÓN DE TRABAJO

JSX es una extensión de la sintaxis de JavaScript que permite escribir marcado parecido a HTML dentro de un archivo JavaScript (react.dev). Al compilar, cada etiqueta se convierte en una llamada a una función.

JSX parece HTML, pero es JavaScript. Vite convierte cada etiqueta en una llamada a una función (jsxDEV en desarrollo) que crea un objeto. Por eso un componente devuelve **una sola raíz**: una función no devuelve dos objetos sueltos. Si no quieres un div extra, usas un **fragmento** <> </>.

Todas las etiquetas se **cierran**: `<img />`, `<input />`. Y los atributos van en **camelCase**: `className` en vez de class, `htmlFor` en vez de for, `maxLength`.

Las **llaves { }** abren una ventana a JavaScript: dentro van variables, cuentas o llamadas. El atributo style recibe un objeto, por eso lleva doble llave: `style={{ fontSize: 24 }}`.

HTML DEL BOCETO

- class="titulo"
- <img src="copa.svg">
- Dos elementos sueltos

JSX CORRECTO

- className="titulo"
-
- Todo dentro de <> </>

FIG 2.1 El mismo marcado antes y después de aplicar las reglas del JSX.

HTML	JSX	¿Por qué cambia?
`class="titulo"`	`className="titulo"`	class es palabra reservada de JavaScript.
`<label for="apodo">`	`<label htmlFor="apodo">`	for también es palabra reservada.
`<img src="copa.svg">`	`<img src="copa.svg" />`	En JSX toda etiqueta se cierra.
`style="font-size:24px"`	`style={{ fontSize: 24 }}`	style recibe un objeto de JavaScript.
`maxlength="12"`	`maxLength={12}`	Atributos en camelCase; el número va entre llaves.

3 IDEAS PARA LLEVAR

1. **Una sola raíz** — Envuelve todo en un div o en <> </>.
2. **Todo se cierra** — <img />, <input />,
.
3. **Llaves = JavaScript** — {local}, {goles[0]}, {{ fontSize: 24 }}.

RESÚMELO EN UNA FRASE CON TUS PALABRAS

UNA DUDA QUE ME QUEDÓ

PALABRAS CLAVE: JSX · fragmento · className · htmlFor · camelCase · expresión · llaves { } · compilación

E03 Código ejemplo: el marcador de la final en JSX

COMPRENDE +15 XP

Así escribió Lucía el marcador. Lo ejecutamos con React 19.3 y Vite 8.3: abajo ves en qué lo convierte Vite y el HTML real que quedó en el navegador.

src/components/Marcador.jsx — JSX

```jsx
export default function Marcador() {
  const local = 'Byte Club'                                  (1)
  const visita = 'Lag Cero'
  const goles = [2, 1]

  return (                                                   (2)
    <>                                                       (3)
      <h2 className="titulo">Final · Copa Pixel</h2>         (4)
      <img
        src="/copa.svg"
        alt="Trofeo de la Copa Pixel"
      />                                                     (5)
      <p style={{ fontSize: 24 }}>                           (6)
        {local} {goles[0]} - {goles[1]} {visita}             (7)
      </p>
      <label htmlFor="apodo">Tu apodo</label>                (8)
      <input id="apodo" maxLength={12} />
    </>
  )
}
```



ASÍ SE VE Así se ve el marcador en localhost:5173.

Predice antes de ejecutar +15 XP

1. Si cambias goles = [2, 1] por [3, 1], ¿qué texto exacto mostrará el párrafo?

2. Si quitas el fragmento <> </>, ¿crees que Vite compilará? ¿Qué mensaje esperas?

☐ ACERTAMOS ☐ CASI ☐ NOS SORPRENDIÓ

lo que recibe el navegador — JS

```js
// Lo que Vite entrega al navegador (extracto)
_jsxDEV(_Fragment, { children: [
  _jsxDEV("h2", {
    className: "titulo",
    children: "Final · Copa Pixel"
  }, …),
  _jsxDEV("p", {
    style: { fontSize: 24 },
    children: [local, " ", goles[0],
      " - ", goles[1], " ", visita]
  }, …),
  …
] }, …)
```

HTML generado (F12 → Elementos) — HTML

```html
<h2 class="titulo">Final · Copa Pixel</h2>
<img alt="Trofeo de la Copa Pixel"
  src="/copa.svg">
<p style="font-size: 24px;">
  Byte Club 2 - 1 Lag Cero
</p>
<label for="apodo">Tu apodo</label>
<input id="apodo" maxlength="12">
```

E04 Anatomía del código: pieza por pieza

COMPRENDE

#	Fragmento	Qué es	Qué hace y por qué importa
1	`const local = 'Byte Club'`	**Datos**	Variables de JavaScript que el marcado va a mostrar.
2	`return (`	**Paréntesis**	Si el JSX empieza en otra línea, va entre paréntesis.
3	`<> … </>`	**Fragmento**	Agrupa varios elementos sin dejar etiquetas de más en el HTML.
4	`className="titulo"`	**Clase CSS**	En JSX es className; en el navegador llega como class.
5	`/>`	**Etiqueta cerrada**	<img> se cierra con />: en JSX toda etiqueta se cierra.
6	`style={{ fontSize: 24 }}`	**Estilo en línea**	Llave externa: JavaScript. Interna: un objeto. 24 se vuelve 24px.
7	`{local} {goles[0]}`	**Expresiones**	Las llaves insertan valores de JavaScript en el texto.
8	`htmlFor="apodo"`	**Atributo del DOM**	Une la etiqueta con su campo; en el navegador llega como for.

E05 Pregunta a la IA por partes del código

COMPRENDE +20 XP

Pregunta por una línea, no por la solución

+20 XP SI VERIFICAN

P1 ¿Por qué en JSX se escribe **className** y no class? Explícalo con la idea de palabra reservada.

P2 ¿Qué significan las dos llaves de **style={{ fontSize: 24 }}**? ¿Qué pasaría con una sola?

P3 ¿En qué se convierte una etiqueta JSX como **<h2 className="titulo">** cuando Vite compila?

Regla de oro: pregunta por una línea o un concepto, nunca por la misión resuelta. Si la IA te da código, ejecútalo y compáralo con la anatomía (E04). Muchas IA aún proponen create-react-app o class en el JSX: si lo ves, contrasta con react.dev.

FÓRMULA PARA PREGUNTAR POR CÓDIGO · ÚSALA EN P1, P2 Y P3

ROL	FRAGMENTO	PREGUNTA	NIVEL	LÍMITE
Actúa como mentor/a de React 19.	Pega solo la línea que no entiendes.	¿Qué hace y qué pasa si la quito?	Explícalo para alguien que empieza.	No resuelvas mi misión.

Parte del código	Qué dijo la IA (en tus palabras)	¿Coincide con la anatomía?	¿Cómo lo comprobaste?
className		◯ Sí ◯ En parte ◯ No	
Doble llave en style		◯ Sí ◯ En parte ◯ No	
JSX compilado		◯ Sí ◯ En parte ◯ No	

E06 Buenas prácticas: así escriben los pros

APLICA

Fragmento antes que div

Agrupa sin cajas de más.

EVITA

```
return (
  <div>
    <h2>Final</h2>
    <p>2 - 1</p>
  </div>
)
```

Un div que solo agrupa suma una caja al HTML y puede alterar la maquetación.

MEJOR ASÍ

```
return (
  <>
    <h2>Final</h2>
    <p>2 - 1</p>
  </>
)
```

El fragmento agrupa sin dejar rastro en el HTML.

Comillas o llaves

Cada cosa en su lugar.

EVITA

```
<h2 className={"titulo"}>
  {"Final"}
</h2>
```

Funciona, pero las llaves sobran: el texto fijo se lee peor.

MEJOR ASÍ

```
<h2 className="titulo">
  Final
</h2>
```

Comillas para texto fijo; llaves cuando el valor viene de JavaScript.

Siempre alt en las imágenes

Describe lo que se ve.

EVITA

```
<img src="/copa.svg" />
```

Un lector de pantalla no puede describir la copa a quien no la ve.

MEJOR ASÍ

```
<img
  src="/copa.svg"
  alt="Trofeo de la Copa Pixel"
/>
```

WCAG 2.2 pide texto alternativo para toda imagen con significado.

ESCRIBAN EN JSX: <LABEL FOR="PAIS">PAÍS</LABEL>

¿CUÁNDO USARÍAN COMILLAS Y CUÁNDO LLAVES EN UN ATRIBUTO? DEN UN EJEMPLO DE CADA CASO.

E07 El caso: el boceto que no compilaba

APLICA · Lectura

Club Pixel · Pantalla de la final

CASO · ARENA U

Tomás diseñó el marcador de la final en un archivo HTML y Lucía lo pegó tal cual dentro del componente. Vite respondió con un error tras otro: primero «Adjacent JSX elements must be wrapped in an enclosing tag», después una etiqueta img sin cerrar.

Cuando por fin compiló, la consola del navegador mostró un aviso por usar class. Valeria propuso revisar el boceto con las tres reglas del JSX antes de pegar nada, y aprovechar para revisar algo que nadie había mirado: la imagen del trofeo no tenía texto alternativo.

> **«JSX se parece a HTML, pero tiene sus propias reglas: primero se traduce, después se pega.»** — Valeria, Club Pixel

ERRORES	AVISO	REGLAS	IMAGEN
2	1	3	1
que detuvieron la compilación	en la consola por usar class	del JSX que se rompieron	sin texto alternativo

E08 Tips de diseño: marcado que todos pueden usar

APLICA · +10 XP



BOCETO DE TOMÁS El marcador, revisado.

1. **Títulos que describen**
   Cada título dice de qué trata su sección, y van en orden: h1, h2, h3.
   W3C, WCAG 2.2, criterio 2.4.6.

2. **Texto alternativo**
   Toda imagen con significado lleva un alt que la describe.
   W3C, WCAG 2.2, criterio 1.1.1.

3. **Etiqueta visible**
   Cada campo tiene su label unido con htmlFor; el texto de ejemplo no la reemplaza.
   Sherwin, NN/g, «Placeholders in Form Fields Are Harmful», 2014.

4. **Idioma de la página**
   La plantilla de Vite trae lang="en" en index.html: cámbialo a "es" para que se lea bien en voz alta.
   W3C, WCAG 2.2, criterio 3.1.1.

RETO DE DISEÑO · DIBUJEN LA TARJETA DE INSCRIPCIÓN Y MARQUEN LOS TÍTULOS, LA IMAGEN CON SU ALT Y CADA CAMPO CON SU ETIQUETA

E09 Misión: la inscripción, en JSX

APLICA +60 XP

Tomás entregó el HTML de la tarjeta de inscripción. Conviértanlo en un componente JSX válido y pruébenlo en el navegador.

Arena U · Inscripcion.jsx

PASO A PASO

1 **Detecta.** Para cada línea del boceto, anoten qué regla del JSX rompe y cómo quedaría. +15 XP

Línea del boceto	Regla que rompe	Cómo queda en JSX
`<h2 class="titulo">Inscripción</h2>`		
`<img src="/escudo.svg">`		
`<label for="nombre">Equipo</label>`		
`<input id="nombre" maxlength="20">`		

2 **Traduce.** Completen el componente. +20 XP

src/components/Inscripcion.jsx — JSX

```jsx
export default function Inscripcion() {
  const cupos = 13

  return (
    <___>
      <h2 _____="titulo">Inscripción</h2>
      <img src="/escudo.svg" alt="_____" ____
      <label _____="nombre">Equipo</label>
      <input id="nombre" _____={20} />
      <p>Quedan {_____} cupos</p>
    </___>
  )
}
```

3 **Inspecciona.** Abran las herramientas del navegador (F12 → Elementos) y anoten cómo llegó cada atributo. +10 XP

En el código	En el navegador
`className="titulo"`	
`htmlFor="nombre"`	
`maxLength={20}`	

4 **Rompe a propósito.** Hagan cada cambio, anoten el mensaje y deshagan. +15 XP

Cambio a propósito	Mensaje que vieron	¿Qué regla lo explica?
Quitar el fragmento <> </>		
Dejar <img> sin cerrar		
Escribir class en vez de className		

DEFINICIÓN DE TERMINADO · MARQUEN SOLO LO QUE COMPROBARON EN PANTALLA

- [ ] El componente devuelve una sola raíz.
- [ ] Todas las etiquetas están cerradas.
- [ ] Los atributos usan className, htmlFor y camelCase.
- [ ] Los datos variables se muestran con llaves.

Firma del copiloto: revisó y probó

E10 Comodines: quitar o usar ayuda

COMPRUEBA -5 / +10 XP

Quitar opciones -5 XP

Para la **P2**: descarta los nombres que no existen en el DOM; JSX usa el nombre de la propiedad del DOM.

Tachamos: _____ y _____

☐ ☐ USADO

Usar pista -5 XP

Para la **P5**: relee la anatomía (E04), fila 7.

Pista usada en: P___

☐ ☐ USADO

Otro equipo +10 AL QUE AYUDA

Expliquen su razonamiento, no la respuesta. Si el otro equipo aprende, ustedes ganan.

¿Cómo pensaron _____ y por qué?

☐ USADO

E11 Preguntas de evaluación

COMPRUEBA +40 XP

Respondan sin mirar las hojas anteriores. Después, revisen juntos y sumen 5 XP por cada respuesta bien justificada (máximo 40).

P1 ¿Qué es JSX según la documentación de React? OPCIÓN MÚLTIPLE

- ◯ Un lenguaje distinto de JavaScript.
- ◯ Una extensión de la sintaxis de JavaScript para escribir marcado parecido a HTML.
- ◯ Un archivo de estilos.
- ◯ Un servidor web.

P2 ¿Cómo se escribe el atributo class de HTML en JSX? OPCIÓN MÚLTIPLE

- ◯ class
- ◯ cssClass
- ◯ className
- ◯ styleClass

P3 Un componente puede devolver dos etiquetas hermanas sin envolverlas en nada. [V] [F] V/F Y POR QUÉ

P4 Relaciona el HTML con su versión en JSX. RELACIONA

for	<img />
<img>	style={{ color: 'red' }}
maxlength	htmlFor
style="color: red"	maxLength

P5 ¿Qué texto mostrará este párrafo en el navegador?

```
const goles = [2, 1]
<p>{goles[0] + goles[1]} goles</p>
```

RESPUESTA CORTA

P6 Vite muestra este error. ¿Qué regla rompe el código y cómo lo arreglarías?

```
Adjacent JSX elements must be wrapped
in an enclosing tag.
```

RESPUESTA CORTA

Reto bonus · Caza el error

+5 XP DE BONO

src/components/Escudo.jsx — JSX

```
1 export default function Escudo() {
2   return (
3     <div>
4       <img src="/copa.svg" alt="Copa">
5     </div>
6   )
7 }
```

Vite no compila y señala la línea 5: `Expected corresponding JSX closing tag for 'img'`. Más abajo agrega: «Opened here» en la línea 4.

¿QUÉ ETIQUETA CAUSA EL PROBLEMA?

¿POR QUÉ VITE ESPERABA ENCONTRAR </IMG>?

ESCRIBAN LA LÍNEA CORREGIDA

E12 Tendencias 2026: el lugar de React

CONECTA +20 XP

REACT	ADMIRADO	DESEADO	NODE.JS
44,7 %	**52,1 %**	**30,7 %**	**48,7 %**
de quienes respondieron usa React	de quienes lo usan quiere seguir con React	quiere trabajar con React el próximo año	la única tecnología web más usada que React

Fuente: Stack Overflow, Developer Survey 2025, frameworks y tecnologías web.

Así lo hace la industria — PRÁCTICA

- **Transformación moderna.** Desde React 17, el JSX se compila a funciones de react/jsx-runtime.
- **Oxc.** Vite 8 transforma el JSX con Oxc, escrito en Rust: de ahí vienen los mensajes de error de este nivel.
- **React Foundation.** Sus fundadores: Amazon, Callstack, Expo, Huawei, Meta, Microsoft, Software Mansion y Vercel.
- **React Compiler.** En Meta Quest Store, algunas interacciones fueron más de 2,5× más rápidas.

Impacto de la IA — 2026

- OPORTUNIDAD Pega un error de Vite y pide que te explique qué regla del JSX rompe.
- OPORTUNIDAD Pídele traducir un fragmento de HTML a JSX y compáralo con las tres reglas.
- RIESGO Solo el 3 % confía mucho en la precisión de la IA: revisa cada atributo.
- RIESGO Si la IA escribe class o deja etiquetas sin cerrar, Vite y la consola te avisarán.

TU POSTURA ¿Te parece mejor mezclar marcado y lógica en un mismo archivo o separarlos? ¿Por qué?

E13 ¿Qué diría un diseñador front?

REFLEXIONA +10 XP

> **El contenido precede al diseño. Diseñar sin contenido no es diseño, es decoración.**
>
> JZ — Jeffrey Zeldman
> Publicación en Twitter, 2008

TU TURNO · +10 XP

Si alguien llena el marcador de cajas vacías y textos de relleno, ¿qué diría un diseñador front? Escribe su frase.

Lupa del diseñador · criterio	¿Su pantalla lo cumple?	Evidencia o cambio que harían
Cada imagen tiene un texto alternativo útil.	◯ Sí ◯ En parte ◯ No	
Cada campo tiene su etiqueta visible.	◯ Sí ◯ En parte ◯ No	
Los títulos siguen un orden lógico.	◯ Sí ◯ En parte ◯ No	

Semáforo: ¿qué tan seguro estás?	ROJO · AMARILLO · VERDE
Explico qué es JSX y en qué se convierte.	○ ○ ○
Aplico sus reglas: una raíz, etiquetas cerradas y camelCase.	○ ○ ○
Inserto datos con llaves { }.	○ ○ ○

Explícalo en 60 segundos. Expliquen a otro equipo las tres reglas del JSX con un error real de cada una. Si el otro equipo encuentra una cuarta regla en react.dev, ambos ganan el bono. +15 XP

Insignia: Traductor de JSX. Coloréala en el Salón de insignias si tu equipo logra 140 XP o más en este nivel.

MARCADOR DEL NIVEL 1.2

- XP ganados + ___
- Comodines usados − ___
- Bono por ayudar + ___

TOTAL _____ /200

NIVEL 1.3 / 1.4

60 MIN · 200 XP

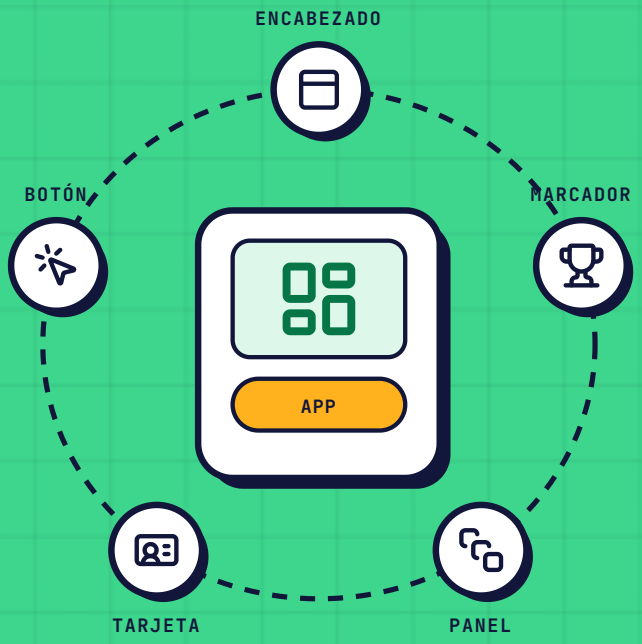
NIVEL 1.3 · PIEZAS QUE ENCAJAN

Componentes: funciones que devuelven interfaz

IDEA CLAVE Componente = función con nombre en mayúscula que devuelve JSX; se exporta, se importa y se usa como <Etiqueta />

TU MISIÓN

App.jsx de Arena U ya tiene 300 líneas: encabezado, marcador y tarjetas, todo mezclado. Cambiar un estilo obliga a buscar en todo el archivo. Tu equipo debe partir la pantalla en **componentes**, cada uno en su archivo, y armarla como un rompecabezas.



RECUERDA SIN MIRAR

Sin mirar el nivel 1.2: ¿por qué un componente debe devolver una sola raíz?

E01 ¿Sabías que? · E02 Explicación · E03 Código ejemplo · E04 Anatomía · E05 Pregunta a la IA · E06 Buenas prácticas · E07 El caso · E08 Tips de diseño · E09 Misión · E10 Comodines · E11 Evaluación · E12 Tendencias · E13 Voz del diseñador

ENGANCHA · COMPRENDE · APLICA · COMPRUEBA · CONECTA · REFLEXIONA

AL TERMINAR ESTE NIVEL PODRÁS

- [ ] Crear componentes como funciones que devuelven JSX.
- [ ] Nombrarlos con mayúscula inicial y usarlos como etiquetas.
- [ ] Exportar e importar componentes entre archivos.
- [ ] Dividir una pantalla en una jerarquía de componentes.

MI META PERSONAL PARA ESTE NIVEL

BOTÍN DEL NIVEL

Reto flash de ¿Sabías que?	+10
Predice el código	+15
Pregunta a la IA verificada	+20
Reto de diseño	+10
Misión paso a paso	+60
Evaluación	+40
Tendencias 2026	+20
Voz del diseñador	+10
Explícalo en 60 segundos	+15
Total del nivel	**200 XP**

E01 ¿Sabías que?

ENGANCHA +10 XP

DISEÑO ATÓMICO

2013

Brad Frost propuso pensar las interfaces como átomos, moléculas y organismos: piezas pequeñas que forman otras mayores.

Fuente: Frost, «Atomic Web Design», 10 jun. 2013.

PASO UNO

1

es el primer paso de «Thinking in React»: dibujar cajas alrededor de cada componente del boceto y ponerles nombre.

Fuente: react.dev, «Thinking in React», 2026.

MAYÚSCULA OBLIGATORIA AB

A–Z

los nombres de los componentes deben empezar con mayúscula; si no, no funcionan.

Fuente: react.dev, «Your First Component», 2026.

RETO FLASH · 60 S — En 60 segundos, dibujen la pantalla de inicio de una app que usen y encierren en cajas las piezas que se repiten. Gana el equipo con más cajas bien nombradas. +10 XP

E02 Explicación breve: piezas con nombre propio

COMPRENDE

DEFINICIÓN DE TRABAJO

Un componente de React es una función de JavaScript que devuelve marcado (JSX). Es una pieza reutilizable de la interfaz: se escribe una vez y se usa como una etiqueta, por ejemplo <TarjetaEquipo />.

Un componente es una función con dos condiciones: su nombre empieza con **mayúscula** y devuelve **JSX**. Con mayúscula, React sabe que <Encabezado /> es tuyo; en minúscula, lo trata como una etiqueta HTML.

Cada componente vive en su archivo y se comparte con **export default**; otro archivo lo trae con **import**. Así App.jsx deja de ser un archivo gigante y se vuelve una lista de piezas.

React propone empezar dibujando cajas sobre el boceto: cada caja es un componente y, idealmente, cada uno se ocupa de **una sola cosa**. Y una regla: nunca definas un componente dentro de otro; decláralos todos en el nivel superior del archivo.

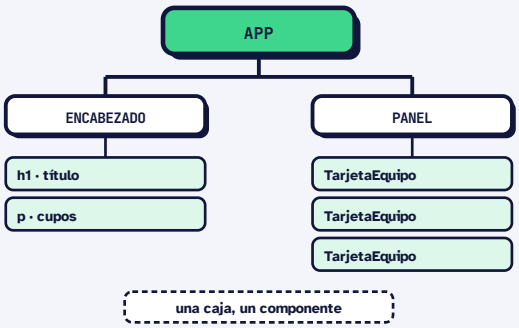


FIG 3.1 La pantalla de Arena U como árbol de componentes: App reúne piezas, y las piezas reúnen etiquetas.

Pregunta	Etiqueta HTML	Componente React
¿Cómo se escribe?	`<header>`	`<Encabezado />`
¿Quién la define?	El navegador.	Tú, con una función.
¿Cómo empieza el nombre?	Con minúscula.	Con mayúscula.
¿Qué dibuja?	Lo que escribas dentro.	El JSX que devuelve la función.
¿Cómo se reutiliza?	Copiando y pegando.	Importándolo y usándolo de nuevo.

3 IDEAS PARA LLEVAR

1. **Función + JSX** — Un componente devuelve interfaz.
2. **Mayúscula** — <Encabezado /> es tuyo; <header>, del navegador.
3. **Exporta e importa** — Cada pieza en su archivo.

RESÚMELO EN UNA FRASE CON TUS PALABRAS

UNA DUDA QUE ME QUEDÓ

PALABRAS CLAVE componente · función · PascalCase · export default · import · jerarquía · reutilizar · nivel superior

E03 Código ejemplo: Arena U en piezas

COMPRENDE +15 XP

Así dividió Lucía la pantalla en tres archivos. Lo ejecutamos con React 19.3 y Vite 8.3, sin avisos en la consola.

src/components/Encabezado.jsx — JSX

```jsx
export default function Encabezado() {
  return (
    <header className="encabezado">
      <h1>Arena U</h1>
      <p>Copa Pixel 2026 · 16 cupos</p>
    </header>
  )
}
```

src/components/TarjetaEquipo.jsx — JSX

```jsx
export default function TarjetaEquipo() {
  return (
    <article className="tarjeta">
      <h3>Los Pixelados</h3>
      <p>5 jugadores</p>
    </article>
  )
}
```

src/App.jsx — JSX

```jsx
import Encabezado from './components/Encabezado.jsx'
import TarjetaEquipo from './components/TarjetaEquipo.jsx'

export default function App() {
  return (
    <>
      <Encabezado />
      <TarjetaEquipo />
      <TarjetaEquipo />
    </>
  )
}
```

carpetas — CARPETAS

```
src/
├── main.jsx
├── App.jsx
└── components/
    ├── Encabezado.jsx
    └── TarjetaEquipo.jsx
```



ASÍ SE VE Dos tarjetas idénticas: la pieza todavía trae los datos escritos por dentro.

Predice antes de ejecutar ☐ ACERTAMOS ☐ CASI ☐ NOS SORPRENDIÓ +15 XP

1. Si agregas un tercer <TarjetaEquipo /> en App.jsx, ¿qué equipo mostrará? ¿Por qué?
2. ¿Cómo harías para que cada tarjeta muestre un equipo distinto sin copiar el archivo?

E04 Anatomía del código: pieza por pieza

COMPRENDE

#	Fragmento	Qué es	Qué hace y por qué importa
1	`export default function Encabezado()`	**Declaración**	Una función con nombre en mayúscula: eso la vuelve componente.
2	`return ( <header> … )`	**Devuelve JSX**	Lo que devuelve la función es lo que React dibuja.
3	`<article className="tarjeta">`	**Raíz de la pieza**	Cada componente tiene su propia raíz y su clase CSS.
4	`<h3>Los Pixelados</h3>`	**Datos fijos**	El equipo está escrito a mano: siempre mostrará lo mismo.
5	`import Encabezado from '…'`	**Importación**	Trae el componente desde su archivo para poder usarlo.
6	`export default function App()`	**Componente raíz**	App reúne las piezas; main.jsx lo dibuja dentro de #root.
7	`<Encabezado />`	**Uso**	El componente se usa como una etiqueta que se cierra sola.
8	`<TarjetaEquipo /> ×2`	**Reutilización**	La misma pieza dos veces: React llama dos veces a la función.

E05 Pregunta a la IA por partes del código

COMPRENDE +20 XP

Pregunta por una línea, no por la solución

+20 XP SI VERIFICAN

P1 ¿Por qué los componentes de React deben empezar con **mayúscula**? ¿Qué hace React con **<tarjetaEquipo />** en minúscula?

P2 ¿Qué diferencia hay entre **export default** y un export con nombre? ¿Cuál usa la plantilla de Vite en App.jsx?

P3 ¿Por qué react.dev dice que nunca se debe **definir un componente dentro de otro**?

Regla de oro: pregunta por una línea o un concepto, nunca por la misión resuelta. Si la IA te da código, ejecútalo y compáralo con la anatomía (E04). Muchas IA aún proponen create-react-app o class en el JSX: si lo ves, contrasta con react.dev.

FÓRMULA PARA PREGUNTAR POR CÓDIGO · ÚSALA EN P1, P2 Y P3

ROL	FRAGMENTO	PREGUNTA	NIVEL	LÍMITE
Actúa como mentor/a de React 19.	Pega solo la línea que no entiendes.	¿Qué hace y qué pasa si la quito?	Explícalo para alguien que empieza.	No resuelvas mi misión.

Parte del código	Qué dijo la IA (en tus palabras)	¿Coincide con la anatomía?	¿Cómo lo comprobaste?
Mayúscula inicial		◯ Sí ◯ En parte ◯ No	
export default		◯ Sí ◯ En parte ◯ No	
Componentes anidados		◯ Sí ◯ En parte ◯ No	

E06 Buenas prácticas: así escriben los pros

APLICA

AB Nombres en PascalCase

Mayúscula al inicio.

EVITA

```
function tarjetaEquipo() { … }
<tarjetaEquipo />
```

React la trata como una etiqueta HTML desconocida y no dibuja nada.

MEJOR ASÍ

```
function TarjetaEquipo() { … }
<TarjetaEquipo />
```

React reconoce el componente y llama a la función.

Cada pieza en su archivo

Un componente por archivo.

EVITA

```
// App.jsx con 300 líneas:
// encabezado, marcador
// y tarjetas, todo junto
```

Cada cambio obliga a buscar en todo el archivo.

MEJOR ASÍ

```
// components/Encabezado.jsx
// components/Marcador.jsx
// components/TarjetaEquipo.jsx
```

Cada archivo se ocupa de una sola cosa.

Nunca anides definiciones

Declara cada componente arriba.

EVITA

```
function App() {
  function Tarjeta() { … }
  return <Tarjeta />
}
```

react.dev advierte que esto es muy lento y causa errores.

MEJOR ASÍ

```
function Tarjeta() { … }

function App() {
  return <Tarjeta />
}
```

Cada componente, en el nivel superior del archivo.

DIBUJEN LAS CAJAS DE LA PANTALLA DE ARENA U Y PONGAN NOMBRE A CADA COMPONENTE.

¿QUÉ COMPONENTE DE ARENA U USARÍAN MÁS DE UNA VEZ? ¿POR QUÉ ESE?

E07 El caso: el archivo de 300 líneas

APLICA Lectura

Club Pixel · Pantalla de equipos

CASO · ARENA U

Lucía escribió toda la pantalla de Arena U en App.jsx. Cuando Tomás pidió cambiar el estilo de las tarjetas de equipo, había ocho copias del mismo marcado y Lucía solo cambió siete: la tarjeta de la final quedó con el diseño viejo y nadie lo notó hasta el día del torneo.

Samuel intentó ayudar creando un componente llamado tarjetaEquipo, en minúscula, y la pantalla quedó vacía. Valeria propuso parar y dibujar cajas sobre el boceto de Tomás: cada caja, un componente con nombre en mayúscula y su propio archivo.

> **«Si una pieza se repite, se escribe una vez.»** — Valeria, Club Pixel

LÍNEAS	COPIAS	OLVIDADA	PIEZAS
300	8	1	5
en un solo App.jsx	del mismo marcado de tarjeta	tarjeta con el diseño viejo	componentes en la nueva versión

E08 Tips de diseño: piezas consistentes

APLICA +10 XP



BOCETO DE TOMÁS La lista de equipos, en piezas.

1. **Átomos, moléculas, organismos**
   Un botón (átomo) forma parte de una tarjeta (molécula), y varias tarjetas forman una lista (organismo).
   Frost, «Atomic Web Design», 2013.

2. **De abajo hacia arriba**
   Primero se construyen los componentes y al final se arman las pantallas.
   componentdriven.org, 2026.

3. **Consistencia**
   Las personas no deberían preguntarse si dos cosas distintas significan lo mismo: misma función, misma pieza.
   Nielsen, heurística 4, 1994 (rev. 2024).

4. **Tokens de diseño**
   Colores y medidas con nombre, como color-primario, que todas las piezas comparten.
   W3C Design Tokens CG, versión estable, 2025.

RETO DE DISEÑO · HAGAN EL INVENTARIO DE PIEZAS DE ARENA U

Pieza	¿Átomo, molécula u organismo?	¿Dónde se repite?
Botón «Inscribir»		
Ícono de equipo		
Tarjeta de equipo		
Panel de equipos		
Encabezado		
Marcador		

E09 Misión: divide y vencerás

APLICA +60 XP

Partan la pantalla de Arena U en componentes y armen App.jsx como un rompecabezas. Turnen piloto y copiloto.

Arena U · pantalla en piezas

PASO A PASO

1 **Dibuja cajas.** Sobre el boceto, decidan qué componentes existen y qué muestra cada uno. +15 XP

Componente	Archivo	¿Qué muestra?
Encabezado		
Marcador		
TarjetaEquipo		

2 **Crea la pieza.** Completen el componente del marcador. +15 XP

src/components/Marcador.jsx — JSX

```jsx
export ________ function ________() {
  return (
    <section className="marcador">
      <h2>Final · Copa Pixel</h2>
      <p>Byte Club 2 – 1 Lag Cero</p>
    </_________>
  )
}
```

3 **Arma la pantalla.** Importen y usen las piezas en App.jsx. +15 XP

src/App.jsx — JSX

```jsx
import Encabezado from './components/Encabezado.jsx'
import ____________ from './components/____________.jsx'

export default function App() {
  return (
    <>
      <____________ />
      <Marcador _____
    </>
  )
}
```

4 **Prueba.** Hagan cada cambio, anoten qué pasó y deshagan. +15 XP

Cambio a propósito	¿Qué pasó?	¿Por qué?
Escribir <marcador /> en minúscula		
Quitar export default de Marcador		
Usar <Marcador /> tres veces		

DEFINICIÓN DE TERMINADO · MARQUEN SOLO LO QUE COMPROBARON EN PANTALLA

- [ ] Cada componente vive en su propio archivo.
- [ ] Los nombres empiezan con mayúscula.
- [ ] App.jsx solo importa y arma piezas.
- [ ] Ningún componente está definido dentro de otro.

Firma del copiloto: revisó y probó

E10 Comodines: quitar o usar ayuda

COMPRUEBA -5 / +10 XP

Quitar opciones -5 XP

Para la **P2**: descarta los nombres que empiezan con minúscula o con símbolos.

```
Tachamos: ______ y ______
```

☐ ☐ USADO

Usar pista -5 XP

Para la **P5**: relee la explicación breve (E02), primer párrafo.

```
Pista usada en: P___
```

☐ ☐ USADO

Otro equipo +10 AL QUE AYUDA

Expliquen su razonamiento, no la respuesta. Si el otro equipo aprende, ustedes ganan.

```
¿Cómo pensaron ______ y por qué?
```

☐ USADO

E11 Preguntas de evaluación

COMPRUEBA +40 XP

Respondan sin mirar las hojas anteriores. Después, revisen juntos y sumen 5 XP por cada respuesta bien justificada (máximo 40).

P1 ¿Qué es un componente de React? OPCIÓN MÚLTIPLE

- ◯ Un archivo CSS.
- ◯ Una función de JavaScript que devuelve marcado (JSX).
- ◯ Una etiqueta HTML nueva del navegador.
- ◯ Un servidor pequeño.

P2 ¿Qué nombre de componente es correcto? OPCIÓN MÚLTIPLE

- ◯ tarjetaEquipo
- ◯ tarjeta-equipo
- ◯ TarjetaEquipo
- ◯ _tarjeta

P3 Es buena práctica definir un componente dentro de la función de otro componente. V F V/F Y POR QUÉ

P4 Ordena el camino de un componente nuevo. ORDENA

- ☐ Exportarla con export default
- ☐ Crear la función en su archivo
- ☐ Usarla como etiqueta
- ☐ Importarla en App.jsx
- ☐ Ver el resultado en el navegador

P5 La consola muestra este aviso. ¿Qué cambiarías en el código?

```
The tag <tarjetaEquipo> is unrecognized
in this browser. If you meant to render a
React component, start its name with an
uppercase letter.
```

RESPUESTA CORTA

P6 Explica con tus palabras por qué una de las ocho tarjetas del caso quedó con el diseño viejo. RESPUESTA CORTA

Reto bonus · Caza el error

+5 XP DE BONO

src/App.jsx — JSX

```jsx
function Encabezado() {
  return
    <h1>Arena U</h1>
}

export default function App() {
  return <Encabezado />
}
```

Vite compila sin errores y la consola no dice nada, pero la pantalla queda vacía. Oxlint, el revisor de la plantilla, sí avisa: `eslint(no-unreachable): Unreachable code.`

¿QUÉ LÍNEA CAUSA EL PROBLEMA?

¿POR QUÉ EL JSX NUNCA LLEGA A DEVOLVERSE?

ESCRIBAN EL RETURN CORREGIDO

E12 Tendencias 2026: todo es un componente

CONECTA +20 XP

NEXT.JS
20,8 %
usa Next.js, un framework basado en React

ANGULAR
18,2 %
usa Angular, que también organiza la interfaz en componentes

VUE.JS
17,6 %
usa Vue.js, con componentes y plantillas

SVELTE
7,2 %
usa Svelte, que compila sus componentes a JavaScript

Fuente: Stack Overflow, Developer Survey 2025, frameworks y tecnologías web.

Así lo hace la industria — PRÁCTICA

- **Design tokens.** Desde octubre de 2025 hay una especificación estable para compartir colores y medidas entre diseño y código.
- **Thinking in React.** La guía oficial empieza dividiendo el boceto en una jerarquía de componentes.
- **React Compiler.** Optimiza componentes y hooks al compilar, sin reescribirlos.
- **Fragment Refs.** React 19.3 permite pasar un ref a un Fragment para manejar un grupo de elementos.

Impacto de la IA — 2026

- OPORTUNIDAD Describe una pantalla y pide una propuesta de componentes; compárala con tus cajas.
- OPORTUNIDAD Pídele revisar si un componente hace demasiadas cosas a la vez.
- RIESGO Puede definir componentes dentro de otros, algo que react.dev desaconseja.
- RIESGO Puede duplicar marcado en vez de reutilizar una pieza que ya existe.

TU POSTURA ¿Qué pieza de una app que usas crees que es el mismo componente repetido muchas veces?

E13 ¿Qué diría un diseñador front?

REFLEXIONA +10 XP

> **No estamos diseñando páginas; estamos diseñando sistemas de componentes.**
>
> SH — Stephen Hay
> Citado por Brad Frost en «Atomic Web Design», 2013

TU TURNO · +10 XP

Si alguien copia y pega la misma tarjeta ocho veces, ¿qué diría un diseñador front? Escribe su frase.

Lupa del diseñador · criterio	¿Su pantalla lo cumple?	Evidencia o cambio que harían
Cada pieza se ocupa de una sola cosa.	◯ Sí ◯ En parte ◯ No	
Las piezas que significan lo mismo se ven igual.	◯ Sí ◯ En parte ◯ No	
La pantalla se arma con piezas, no con copias.	◯ Sí ◯ En parte ◯ No	

Semáforo: ¿qué tan seguro estás? — ROJO · AMARILLO · VERDE

Creo componentes como funciones que devuelven JSX.	◯ ◯ ◯
Los nombro con mayúscula y los uso como etiquetas.	◯ ◯ ◯
Exporto e importo componentes entre archivos.	◯ ◯ ◯

Explícalo en 60 segundos. Expliquen a otro equipo cómo pasaron de un archivo de 300 líneas a piezas con nombre. Si el otro equipo encuentra una pieza más en su boceto, ambos ganan el bono. +15 XP

Insignia: Armador de piezas. Coloréala en el Salón de insignias si tu equipo logra 140 XP o más en este nivel.

MARCADOR DEL NIVEL 1.3

XP ganados + ___
Comodines usados − ___
Bono por ayudar + ___

TOTAL _____ /200

NIVEL 1.4 · PIEZAS QUE RECIBEN DATOS

Props: datos de padre a hijo

IDEA CLAVE Props = datos que el padre le pasa al hijo como atributos; el hijo los recibe en un objeto y no los cambia

TU MISIÓN

Arena U ya tiene 16 equipos inscritos, pero TarjetaEquipo siempre muestra «Los Pixelados»: los datos están escritos dentro de la pieza. Tu equipo debe enseñarle a la tarjeta a **recibir datos** con props, para que una sola pieza sirva para todos.



RECUERDA SIN MIRAR

Sin mirar el nivel 1.3: ¿por qué el nombre de un componente debe empezar con mayúscula?

E01 ¿Sabías que? · E02 Explicación · E03 Código ejemplo · E04 Anatomía · E05 Pregunta a la IA · E06 Buenas prácticas · E07 El caso · E08 Tips de diseño · E09 Misión · E10 Comodines · E11 Evaluación · E12 Tendencias · E13 Voz del diseñador

ENGANCHA · COMPRENDE · APLICA · COMPRUEBA · CONECTA · REFLEXIONA

AL TERMINAR ESTE NIVEL PODRÁS

- [ ] Explicar qué son las props y en qué se parecen a los parámetros.
- [ ] Pasar texto, números y booleanos de padre a hijo.
- [ ] Recibir props desestructurando y con valores por defecto.
- [ ] Usar children para anidar contenido dentro de una pieza.

MI META PERSONAL PARA ESTE NIVEL

BOTÍN DEL NIVEL

Reto flash de ¿Sabías que?	+10
Predice el código	+15
Pregunta a la IA verificada	+20
Reto de diseño	+10
Misión paso a paso	+60
Evaluación	+40
Tendencias 2026	+20
Voz del diseñador	+10
Explícalo en 60 segundos	+15
Total del nivel	**200 XP**

E01 ¿Sabías que?

ENGANCHA +10 XP

UN ARGUMENTO

1

solo argumento recibe un componente: el objeto props, con todo lo que el padre le pasó.

Fuente: react.dev, «Passing Props to a Component», 2026.

SOLO LECTURA

TypeError

lanzó el navegador cuando una tarjeta intentó cambiar su prop nombre en modo desarrollo.

Fuente: Prueba propia con React 19.3 y Chromium, 7 oct. 2026.

PROP ESPECIAL

children

es la prop que recibe todo lo que el padre escribe entre la etiqueta de apertura y la de cierre.

Fuente: react.dev, «Passing Props to a Component», 2026.

RETO FLASH · 60 S — En 60 segundos, escriban funciones que reciban datos, como sumar(2, 3) o saludar("Ana"), y digan qué parámetros tiene cada una. Gana el equipo con más ejemplos correctos. +10 XP

E02 Explicación breve: los parámetros de una pieza

COMPRENDE

DEFINICIÓN DE TRABAJO

Las props son la información que un componente padre le pasa a un componente hijo, como los argumentos que recibe una función. Se escriben como atributos de JSX y el hijo las recibe en un solo objeto.

En JavaScript, `sumar(2, 3)` recibe datos por sus parámetros. Un componente hace lo mismo: el padre escribe **atributos** en la etiqueta, como `<TarjetaEquipo nombre="Byte Club" />`, y React se los entrega al hijo en el objeto **props**.

El hijo suele **desestructurar** ese objeto en la firma: `({ nombre, jugadores })`. Si una prop puede faltar, se le da un **valor por defecto**: `enVivo = false`. El texto va entre comillas; números, booleanos y variables, entre llaves.

Las props viajan en una sola dirección, **de padre a hijo**, y son de **solo lectura**: el hijo las usa, pero no las cambia. Lo que el padre escribe entre la etiqueta de apertura y la de cierre llega en una prop especial: **children**.

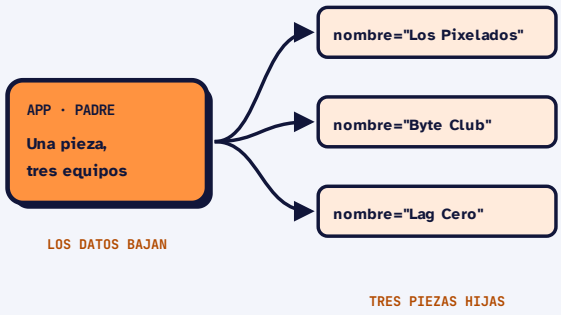


FIG 4.1 El padre usa la misma pieza tres veces y le pasa a cada una un nombre distinto: los datos bajan, nunca suben.

Quieres pasar	Así lo escribe el padre	Así llega al hijo
Texto	`nombre="Byte Club"`	`'Byte Club'`
Número	`jugadores={4}`	`4`
Booleano verdadero	`enVivo`	`true`
Nada (no la escribe)	`<TarjetaEquipo />`	`undefined` o el valor por defecto
Contenido anidado	`<Panel>…</Panel>`	`children`

3 IDEAS PARA LLEVAR

1. **Bajan del padre** — El padre escribe atributos; el hijo los recibe.
2. **Un solo objeto** — Se desestructura: ({ nombre, jugadores }).
3. **Solo lectura** — El hijo usa sus props, nunca las cambia.

RESÚMELO EN UNA FRASE CON TUS PALABRAS

UNA DUDA QUE ME QUEDÓ

PALABRAS CLAVE: props · padre · hijo · atributo · desestructurar · valor por defecto · children · solo lectura

E03 Código ejemplo: una tarjeta, muchos equipos

COMPRENDE +15 XP

Así le enseñó Valeria a la tarjeta a recibir datos. Lo ejecutamos con React 19.3 y Vite 8.3, sin avisos en la consola.

src/components/TarjetaEquipo.jsx — JSX

```jsx
export default function TarjetaEquipo({
  nombre,
  jugadores,
  enVivo = false,
}) {
  return (
    <article className="tarjeta">
      <h3>{nombre}</h3>
      <p>{jugadores} jugadores</p>
      <span>
        {enVivo ? 'En vivo' : 'Por jugar'}
      </span>
    </article>
  )
}
```

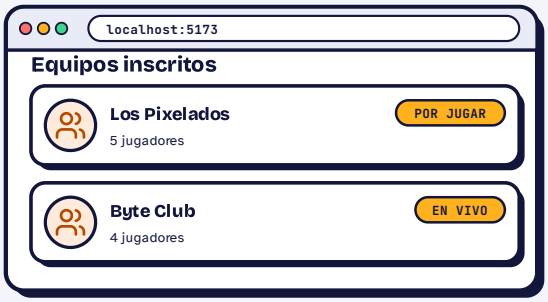
src/components/Panel.jsx — JSX

```jsx
export default function Panel({
  titulo,
  children,
}) {
  return (
    <section className="panel">
      <h2>{titulo}</h2>
      {children}
    </section>
  )
}
```

src/App.jsx — JSX

```jsx
import Panel from './components/Panel.jsx'
import TarjetaEquipo from './components/TarjetaEquipo.jsx'

export default function App() {
  return (
    <Panel titulo="Equipos inscritos">
      <TarjetaEquipo
        nombre="Los Pixelados"
        jugadores={5}
      />
      <TarjetaEquipo
        nombre="Byte Club"
        jugadores={4}
        enVivo
      />
    </Panel>
  )
}
```



ASÍ SE VE La misma pieza, dos equipos: cada tarjeta muestra los datos que recibió.

Predice antes de ejecutar ☐ ACERTAMOS ☐ CASI ☐ NOS SORPRENDIÓ +15 XP

1. Si borras la línea enVivo en App.jsx, ¿qué texto mostrará la tarjeta de Byte Club?

2. Si escribes jugadores="4" en vez de jugadores={4}, ¿cambia lo que ves? ¿Y si la tarjeta mostrara {jugadores + 1}?

E04 Anatomía del código: pieza por pieza

COMPRENDE

#	Fragmento	Qué es	Qué hace y por qué importa
1	`function TarjetaEquipo({ … })`	**Recibe props**	Las llaves sacan nombre, jugadores y enVivo del objeto props.
2	`enVivo = false,`	**Valor por defecto**	Si el padre no la escribe, vale false.
3	`{nombre}`	**Usa la prop**	Se muestra entre llaves, como cualquier variable.
4	`{enVivo ? … : …}`	**Decide con una prop**	El operador ternario elige el texto según el booleano.
5	`{children}`	**Contenido anidado**	Aquí aparece lo que App escribe dentro de <Panel>.
6	`<Panel titulo="…">`	**Prop de texto**	Texto fijo entre comillas; además, envuelve a las tarjetas.
7	`jugadores={5}`	**Prop numérica**	Números, variables y objetos van entre llaves.
8	`enVivo`	**Booleano corto**	Escribir solo el nombre equivale a enVivo={true}.

E05 Pregunta a la IA por partes del código

COMPRENDE +20 XP

Pregunta por una línea, no por la solución

+20 XP SI VERIFICAN

P1 ¿Por qué se dice que las props son como los **argumentos de una función**? Dame un ejemplo que no sea mi misión.

P2 ¿Qué diferencia hay entre **jugadores="5"** y **jugadores={5}**? ¿Qué pasa si después sumo 1?

P3 ¿Qué es la prop **children** y cuándo conviene usarla en vez de una prop con nombre?

Regla de oro: pregunta por una línea o un concepto, nunca por la misión resuelta. Si la IA te da código, ejecútalo y compáralo con la anatomía (E04). Muchas IA aún proponen create-react-app o class en el JSX: si lo ves, contrasta con react.dev.

FÓRMULA PARA PREGUNTAR POR CÓDIGO · ÚSALA EN P1, P2 Y P3

ROL	FRAGMENTO	PREGUNTA	NIVEL	LÍMITE
Actúa como mentor/a de React 19.	Pega solo la línea que no entiendes.	¿Qué hace y qué pasa si la quito?	Explícalo para alguien que empieza.	No resuelvas mi misión.

Parte del código	Qué dijo la IA (en tus palabras)	¿Coincide con la anatomía?	¿Cómo lo comprobaste?
Props y argumentos		◯ Sí ◯ En parte ◯ No	
Comillas o llaves		◯ Sí ◯ En parte ◯ No	
children		◯ Sí ◯ En parte ◯ No	

E06 Buenas prácticas: así escriben los pros

APLICA

Desestructura en la firma

Qué datos usa, a la vista.

EVITA

```
function TarjetaEquipo(props) {
  return <h3>{props.nombre}</h3>
}
```

Funciona, pero hay que leer todo el cuerpo para saber qué props usa.

MEJOR ASÍ

```
function TarjetaEquipo({ nombre }) {
  return <h3>{nombre}</h3>
}
```

La primera línea dice qué datos necesita la pieza.

Números entre llaves

Comillas solo para texto.

EVITA

```
<TarjetaEquipo jugadores="5" />
// {jugadores + 1} → "51"
```

Con comillas llega el texto "5": al sumar 1, JavaScript une textos.

MEJOR ASÍ

```
<TarjetaEquipo jugadores={5} />
// {jugadores + 1} → 6
```

Con llaves llega el número 5 y la suma funciona.

No cambies tus props

Crea una variable nueva.

EVITA

```
function TarjetaEquipo(props) {
  props.nombre =
    props.nombre.toUpperCase()
}
```

En desarrollo, React lanzó TypeError: la prop es de solo lectura.

MEJOR ASÍ

```
function TarjetaEquipo({ nombre }) {
  const titulo = nombre.toUpperCase()
  return <h3>{titulo}</h3>
}
```

La prop queda intacta; el cambio vive en titulo.

ESCRIBAN CÓMO APP LE PASARÍA A TARJETAEQUIPO EL EQUIPO «LAG CERO», CON 3 JUGADORES Y EN VIVO.

¿POR QUÉ CREEN QUE REACT NO DEJA QUE UN HIJO CAMBIE SUS PROPS?

E07 El caso: 16 equipos, una sola tarjeta

APLICA · Lectura

Club Pixel · Lista de inscritos

CASO · ARENA U

Con la inscripción abierta, la Copa Pixel ya tiene 16 equipos. Samuel propuso copiar TarjetaEquipo.jsx dieciséis veces, una por equipo, y cambiar el nombre a mano en cada archivo. Lucía le recordó lo que pasó con las ocho copias del marcado: una quedó desactualizada y nadie lo notó.

Valeria propuso usar props: una sola tarjeta que recibe el nombre, los jugadores y si el equipo juega en vivo. Al probar, Samuel quiso pasar el nombre a mayúsculas cambiando la prop dentro de la tarjeta y la pantalla quedó en blanco. La consola decía: TypeError: Cannot assign to read only property 'nombre'.

> **«Una pieza, muchos datos: el padre decide qué mostrar y el hijo lo dibuja.»** — Valeria, Club Pixel

EQUIPOS	ARCHIVOS	PROPS	ERROR
16	1	3	1
inscritos en la Copa Pixel	TarjetaEquipo.jsx para todos	nombre, jugadores y enVivo	TypeError al cambiar una prop

E08 Tips de diseño: estados que se entienden

APLICA · +10 XP



BOCETO DE TOMÁS Partidos de hoy, con estados.

1. **Estado con texto, no solo con color**
   «En vivo» y «Por jugar» se leen; el color solo acompaña al texto.
   W3C, WCAG 2.2, criterio 1.4.1.

2. **Estados del botón**
   Pequeños cambios visuales distinguen los estados de un botón: se sabe qué se puede tocar y si la acción se registró.
   Gordon, NN/g, «Button States: Communicate Interaction», 2025.

3. **Objetivos de toque**
   Lo que se toca mide al menos 24 × 24 píxeles CSS, salvo excepciones.
   W3C, WCAG 2.2, criterio 2.5.8.

4. **Estados vacíos útiles**
   Si no hay datos, dilo y muestra el siguiente paso: una pantalla en blanco confunde.
   Kaplan, NN/g, «Designing Empty States in Complex Applications», 2021.

RETO DE DISEÑO · DEFINAN CÓMO SE VERÁ CADA ESTADO DE TARJETAPARTIDO

Estado del partido	Texto que se ve	¿Qué color lo acompaña? ¿Por qué?
Por jugar		
En vivo		
Terminado		
Sin partidos hoy		

E09 Misión: una tarjeta, muchos partidos

APLICA +60 XP

Creen TarjetaPartido, una pieza que recibe datos por props, y úsenla para tres partidos de la Copa Pixel. Turnen piloto y copiloto.

Arena U · TarjetaPartido.jsx

PASO A PASO

1 Diseña las props. Decidan qué datos necesita la tarjeta y cómo los escribirá App. +15 XP

Prop	Tipo (texto, número, booleano)	Ejemplo en App.jsx
local		
visita		
hora		
enVivo		

2 Recibe los datos. Completeren el componente. +15 XP

src/components/TarjetaPartido.jsx — JSX

```jsx
export default function TarjetaPartido({ local, visita, hora, enVivo = _____ }) {
  return (
    <article className="partido">
      <h3>{_____} - {_____}</h3>
      <p>Hora: {_____}</p>
      <span>{enVivo ? '_____' : 'Por jugar'}</span>
    </article>
  )
}
```

3 Pasa las props. Usen la tarjeta tres veces en App.jsx, cada una con datos distintos. +15 XP

src/App.jsx — JSX

```jsx
import TarjetaPartido from './components/TarjetaPartido.jsx'

export default function App() {
  return (
    <>
      <TarjetaPartido local="Byte Club" visita="Lag Cero" hora="18:00" _____ />
      <TarjetaPartido local=_____ visita=_____ hora=_____ />
      <_____ _____ />
    </>
  )
}
```

4 Prueba. Hagan cada cambio, anoten qué pasó y deshagan. +15 XP

Cambio a propósito	¿Qué pasó?	¿Por qué?
Quitar la prop hora de un partido		
Escribir enVivo={false} y luego solo enVivo		
Cambiar una prop dentro de TarjetaPartido		

DEFINICIÓN DE TERMINADO · MARQUEN SOLO LO QUE COMPROBARON EN PANTALLA

- [ ] Existe un solo archivo TarjetaPartido.jsx.
- [ ] Cada partido se ve distinto solo por sus props.
- [ ] Los números y booleanos van entre llaves.
- [ ] Ninguna prop se modifica dentro del hijo.

Firma del copiloto: revisó y probó

E10 Comodines: quitar o usar ayuda

COMPRUEBA · -5 / +10 XP

Quitar opciones — -5 XP

Para la **P2**: relee la anatomía (E04), fila 8.

> Tachamos: _____ y _____

☐ ☐ USADO

Usar pista — -5 XP

Para la **P5**: relee la tabla de la explicación breve (E02), fila «Nada».

> Pista usada en: P___

☐ ☐ USADO

Otro equipo — +10 AL QUE AYUDA

Expliquen su razonamiento, no la respuesta. Si el otro equipo aprende, ustedes ganan.

> ¿Cómo pensaron _____ y por qué?

☐ USADO

E11 Preguntas de evaluación

COMPRUEBA · +40 XP

Respondan sin mirar las hojas anteriores. Después, revisen juntos y sumen 5 XP por cada respuesta bien justificada (máximo 40).

P1 ¿A qué se parecen las props de un componente? OPCIÓN MÚLTIPLE

- ◯ A los estilos CSS de una página.
- ◯ A los argumentos que recibe una función.
- ◯ A una base de datos.
- ◯ A un archivo de configuración.

P2 ¿Qué valor recibe enVivo en `<TarjetaEquipo enVivo />`? OPCIÓN MÚLTIPLE

- ◯ undefined
- ◯ "enVivo"
- ◯ true
- ◯ false

P3 Un componente hijo puede cambiar el valor de sus props para actualizar la pantalla. V F V/F Y POR QUÉ

P4 Relaciona cada dato con la forma de pasarlo. RELACIONA

Texto fijo	enVivo
Número	<Panel>…</Panel>
Booleano verdadero	nombre="Byte Club"
Contenido anidado	jugadores={4}

P5 App escribe `<TarjetaEquipo />` sin nombre. ¿Qué cambiarías en TarjetaEquipo para que muestre «Sin nombre»? RESPUESTA CORTA

P6 Explica con tus palabras por qué 16 copias de TarjetaEquipo.jsx serían un problema. RESPUESTA CORTA

Reto bonus · Caza el error

+5 XP DE BONO

src/App.jsx — JSX

```jsx
function TarjetaEquipo(nombre) {
  return <h3>{nombre}</h3>
}

export default function App() {
  return (
    <TarjetaEquipo nombre="Los Pixelados" />
  )
}
```

Vite compila, pero la pantalla queda vacía y la consola muestra: `Objects are not valid as a React child (found: object with keys {nombre}).`

¿QUÉ RECIBE EN REALIDAD EL PARÁMETRO NOMBRE?

¿POR QUÉ REACT NO PUEDE DIBUJAR ESE VALOR?

ESCRIBAN LA PRIMERA LÍNEA CORREGIDA

E12 Tendencias 2026: programar con IA a tu lado

CONECTA +20 XP

USO
84 %
usa o planea usar herramientas de IA para programar

DESCONFIANZA
46 %
desconfía de la precisión de lo que produce la IA

AGENTES
31 %
ya usa agentes de IA para programar

DEPURAR
45,2 %
dice que depurar código hecho por IA le toma más tiempo

Fuente: Stack Overflow, Developer Survey 2025, sección de IA.

Así lo hace la industria — PRÁCTICA

- **Aprender con sandboxes.** «Passing Props to a Component», de react.dev, trae ejemplos que editas y ejecutas en el navegador.
- **Spread con medida.** Con `{...props}` se reenvían todas las props; react.dev recomienda usarlo con moderación.
- **Props con tipos.** create-vite ofrece la plantilla react-ts: con TypeScript, el editor avisa si falta una prop.
- **Probado en este cuaderno.** En modo desarrollo, cambiar una prop lanzó TypeError: React protege las props.

Impacto de la IA — 2026

- OPORTUNIDAD Pídele ejemplos de props para un componente y compáralos con la tabla de E02.
- OPORTUNIDAD Pega un error como «Objects are not valid as a React child» y pide que te explique la causa.
- RIESGO Puede sugerir cambiar una prop dentro del hijo, algo que React no permite.
- RIESGO Puede pasar todo con `{...props}` sin dejar claro qué datos usa la pieza.

TU POSTURA ¿Qué datos de una app que usas crees que llegan como props a sus tarjetas?

E13 ¿Qué diría un diseñador front?

REFLEXIONA +10 XP

> **El diseño siempre debe mantener informadas a las personas sobre lo que está pasando, con retroalimentación adecuada y en un tiempo razonable.**
>
> JN — Jakob Nielsen
> Heurística 1, «10 Usability Heuristics», 1994 (rev. 2024)

TU TURNO · +10 XP

Si una tarjeta no dice si el partido está en vivo o por jugar, ¿qué diría un diseñador front? Escribe su frase.

Lupa del diseñador · criterio	¿Su pantalla lo cumple?	Evidencia o cambio que harían
Cada tarjeta dice con texto en qué estado está.	◯ Sí ◯ En parte ◯ No	
La misma pieza se ve igual con datos distintos.	◯ Sí ◯ En parte ◯ No	
Si no hay datos, la pantalla lo dice.	◯ Sí ◯ En parte ◯ No	

Semáforo: ¿qué tan seguro estás?	ROJO · AMARILLO · VERDE
Paso datos de padre a hijo con props.	◯ ◯ ◯
Recibo props desestructurando y con valores por defecto.	◯ ◯ ◯
Uso children para anidar contenido.	◯ ◯ ◯

Explícalo en 60 segundos. Expliquen a otro equipo cómo una sola TarjetaEquipo sirve para 16 equipos. Si el otro equipo pasa una prop nueva y funciona a la primera, ambos ganan el bono. **+15 XP**

Insignia: Mensajero de props. Coloréala en el Salón de insignias si tu equipo logra 140 XP o más en este nivel.

MARCADOR DEL NIVEL 1.4

XP ganados + ___
Comodines usados − ___
Bono por ayudar + ___

TOTAL _____ /200

Salón de insignias

Cierre de ruta

Colorea cada insignia cuando tu equipo alcance 140 XP o más en el nivel. La corona se gana al llegar al último rango.



Primer arranque
Nivel 1.1
☐ ≥ 140 XP



Traductor de JSX
Nivel 1.2
☐ ≥ 140 XP



Armador de piezas
Nivel 1.3
☐ ≥ 140 XP



Mensajero de props
Nivel 1.4
☐ ≥ 140 XP



Pensador/a en componentes
Ruta completa
☐ ≥ 640 XP

MI MAPA DE PROGRESO

Nivel	Tema	XP	Comodines	Semáforo final
1.1	**React y Vite: tu primer proyecto**			
1.2	**JSX: HTML dentro de JavaScript**			
1.3	**Componentes: funciones que devuelven interfaz**			
1.4	**Props: datos de padre a hijo**			

RETO FINAL · ARENA U EN PIEZAS: UNAN LOS CUATRO SUBNIVELES EN UNA PANTALLA

Pieza	Lo que deben construir (sin mirar las misiones)	Repasa	Hecho
Proyecto	Un proyecto nuevo con create-vite que corra en localhost.	Nivel 1.1	☐
JSX	Un encabezado con imagen, alt, className y un dato entre llaves.	Nivel 1.2	☐
Componentes	Encabezado, Panel y TarjetaPartido, cada uno en su archivo.	Nivel 1.3	☐
Props	Tres partidos con la misma tarjeta y datos distintos.	Nivel 1.4	☐
Diseño	Textos que se escanean, estados con texto y contraste suficiente.	E08	☐

MI PLANO FINAL · 3 · 2 · 1

3 IDEAS QUE ME LLEVO

2 COSAS QUE APLICARÉ EN UN PROYECTO

1 PREGUNTA QUE AÚN TENGO

Fuentes y referencias

Para investigar más

Datos, frases y noticias vienen de estas fuentes; las frases en inglés se tradujeron al español. Arena U, el Club Pixel, la Universidad Altamira y sus personajes son ficticios.

DOCUMENTACIÓN OFICIAL

React (2026). *react.dev*: «Creating a React App», «Your First Component», «Writing Markup with JSX», «Passing Props to a Component», «Thinking in React» y «StrictMode».

Vite (2026). *vite.dev*: «Getting Started» (documentación de Vite 8.3). Registro de npm: react 19.3.0, vite 8.3.3, create-vite 9.2.1 y @vitejs/plugin-react 6.1.2 (consulta del 7 oct. 2026).

W3C (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*: criterios 1.1.1, 1.4.1, 1.4.3, 2.4.6, 2.5.8 y 3.1.1. W3C Design Tokens Community Group (2025, 28 oct.). Primera versión estable de la especificación.

ANUNCIOS DEL EQUIPO DE REACT Y DE VITE

Ruan, L. (2020, 22 sep.). «Introducing the New JSX Transform». React Blog. React Blog (2025, 14 feb.). «Sunsetting Create React App»; (2025, 7 oct.). «React Compiler v1.0»; (2026, 24 feb.). «The React Foundation: A New Home for React Hosted by the Linux Foundation»; (2026, 9 sep.). «React 19.3».

Vite Blog (2026, 12 mar.). «Announcing Vite 8». Wikipedia (2026). «React (software)» y «Vite (software)».

DISEÑO DE INTERFACES Y EXPERIENCIA DE USUARIO

Frost, B. (2013, 10 jun.). «Atomic Web Design». bradfrost.com (cita de Stephen Hay). Component Driven (2026). componentdriven.org.

Krug, S. (2000). *Don't Make Me Think*. New Riders. Wroblewski, L. (2011). *Mobile First*. A Book Apart.

Nielsen, J. (1994, rev. 2024). «10 Usability Heuristics for User Interface Design»; (1997). «How Users Read on the Web». Nielsen Norman Group.

Sherwin, K. (2014). «Placeholders in Form Fields Are Harmful»; Kaplan, K. (2021). «Designing Empty States in Complex Applications»; Gordon, K. (2025). «Button States: Communicate Interaction». Nielsen Norman Group.

Zeldman, J. (2008). «Content precedes design...». Publicación en Twitter.

ENCUESTAS

Stack Overflow (2025). *Developer Survey*: secciones «Technology» (frameworks web) y «AI».

CÓDIGO DE ESTE CUADERNO

Todo el código se creó y ejecutó el 7 oct. 2026 con create-vite 9.2.1, Vite 8.3.3, React 19.3.0, Node.js 22.22 y Chromium. Las salidas de la terminal, el HTML generado y los mensajes de error se copiaron de esas ejecuciones.

CHULETA DE REACT · LO ESENCIAL DEL NIVEL 1

Subnivel	Clave	Recuerda
1.1 · React y Vite	`npm create vite@latest` `npm install` `npm run dev`	La app abre en localhost:5173.
1.2 · JSX	`<> </>` `className` `<img />` `{valor}`	Una raíz, todo cerrado, camelCase.
1.3 · Componentes	`export default function Tarjeta()` `<Tarjeta />`	Mayúscula inicial; nunca uno dentro de otro.
1.4 · Props	`<Tarjeta nombre="…" jugadores={5} />` `({ nombre, children })`	Bajan del padre al hijo; solo lectura.

PARA SEGUIR APRENDIENDO · RECURSOS GRATUITOS Y OFICIALES

react.dev/learn
Guía oficial con ejemplos que editas en el navegador.

vite.dev/guide
Cómo crear, servir y empaquetar con Vite.

MDN Web Docs
Referencia abierta de HTML, CSS y JavaScript.

REPASO FINAL · MENSAJES REALES QUE YA SABES LEER: ESCRIBAN LA CAUSA SIN MIRAR

Mensaje que apareció en este cuaderno	¿Qué lo causa?	Subnivel
`Target container is not a DOM element.`		1.1
`Adjacent JSX elements must be wrapped in an enclosing tag.`		1.2
`Expected corresponding JSX closing tag for 'img'.`		1.2
`The tag <tarjetaEquipo> is unrecognized in this browser.`		1.3
`eslint(no-unreachable): Unreachable code.`		1.3
`Objects are not valid as a React child`		1.4
`Cannot assign to read only property 'nombre'`		1.4

Créditos de diseño. Tipografías Bricolage Grotesque, Atkinson Hyperlegible, JetBrains Mono y Press Start 2P. Iconos Lucide (licencia ISC). Ilustraciones vectoriales creadas para este cuaderno. Las citas se tradujeron conservando su sentido: consulta la fuente original para citarlas en un trabajo.